SERRA: A Socially-Aware Environmental Robotic Research Assistant for Greenhouse Management

Elective in AI - HRI + RBC Report

Filippo Croce 2147314
Simone Sangiorgio 1886044

Each student contributed equally to the development of the project.

Abstract

This report details the design, implementation, and evaluation of SERRA, a socially-aware robotic assistant for managing a simulated greenhouse in a space station. The project integrates principles of Human-Robot Interaction (HRI) and Behavior-Based Robotics (BBR) to create a system embodied by a TIAGo robot within the Robot Operating System (ROS) framework. SERRA leverages a Large Language Model (LLM) for advanced natural language understanding and dialogue management, a persistent Knowledge Graph (KG) for short and long-term memory and world modeling, and a voice-first interface for intuitive user interaction. The system architecture supports hierarchical task planning, personalized user adaptation, and proactive behaviors. Key innovations include a dynamic persona-generation system that tailors robot communication to user roles and relationship history, a robust voice-input module designed to handle hardware inconsistencies in virtual environments, and a declarative task modeling framework that allows complex behaviors to be triggered by simple natural language commands. The result is a comprehensive proof-of-concept demonstrating the potential for creating intelligent, collaborative robots that can operate effectively alongside humans in complex, semi-structured environments.

Contents

1 Introduction **4**
1.1 Context and Motivation . . . 4
1.2 Objectives . . . 4
1.3 Summary of the Results . . . 5

2 Related Work **5**
2.1 Long-Term Human-Robot Interaction . . . 5
2.2 Benchmarking and Competitions . . . 5
2.3 Conversational AI and LLMs in Robotics . . . 6
2.4 Knowledge Representation and Reasoning in Robotics . . . 6
2.5 Robotics in Precision Agriculture . . . 6

3 System Architecture **7**
3.1 High-Level Overview . . . 7
3.2 Architectural Diagram . . . 7
3.3 Core Components . . . 7

4 Platform Selection and Development Environment **8**
4.1 Robot Platform: PAL Robotics TIAGo . . . 8
4.2 Development Framework: ROS Noetic . . . 8
4.3 Key Software Libraries . . . 9

5 Human-Robot Interaction (HRI) **9**
5.1 User Modeling and Personalization . . . 9
5.1.1 The Structure of a User Profile . . . 9
5.1.2 Dynamic Adaptation . . . 10
5.2 Conversational Dynamics and Persona Generation . . . 10
5.3 Voice-Centric Modality and Robustness . . . 11
5.4 Physical Social Cues . . . 11

6 Robot Benchmarking and Competitions (RBC) 11
6.1 World Modeling: Dynamic vs. Static Knowledge . . . 11
6.1.1 Dynamic State from `environment_publisher.py` . . . 12
6.1.2 Static Knowledge from `kg_manager.py` . . . 12
6.2 Hierarchical Task Planning and Execution . . . 13
6.2.1 The Planning Process . . . 13
6.3 Proactive and Recommendation Behaviors . . . 14
6.4 Navigation and Localization . . . 14

7 Functionality and Task Benchmarks 14
7.1 Functionality Benchmarks (Component-Level Evaluation) . . . 14
7.1.1 Natural Language Understanding (NLU) `llm_manager.py` . . . 15
7.1.2 Knowledge Retrieval - `kg-manager.py` . . . 15
7.1.3 Navigation Performance - ROS `move_base` . . . 15
7.1.4 Speech-to-Text Reliability - `whisper_manager.py` . . . 15
7.1.5 Text-To-Speech Reliability - `speech_manager.py` . . . 16
7.1.6 Recommendation Logic - `llm_manager.py` . . . 16
7.2 Task Benchmark: The ”Greenhouse Inspector” Challenge . . . 16
7.2.1 Task Definition and Rules . . . 17
7.2.2 Defined Achievements . . . 17
7.2.3 Defined personas . . . 17

8 Experiments and Results 17
8.1 Experiment 1: Commander Interaction (Proactivity and Command Precedence) . . . 17
8.1.1 Interaction Log and Analysis . . . 18
8.2 Experiment 2: Scientist Interaction (Data-Driven Recommendation) . . . 18
8.2.1 Interaction Log and Analysis . . . 18
8.3 Experiment 3: Child Interaction (Social Rules and Safety) . . . 19
8.3.1 Interaction Log and Analysis . . . 19
8.4 Experiment 4: Unknown User (Memory Persistence and Keyword Spotting) . . . 20
8.4.1 Interaction Log and Analysis . . . 20
8.4.2 Results and Analysis . . . 21

9 Conclusion and Future Work 21
9.1 Conclusion . . . 21
9.2 Future Work . . . 21

1 Introduction

1.1 Context and Motivation

The field of robotics is undergoing a significant paradigm shift, moving from pre-programmed industrial machines to intelligent, autonomous agents capable of operating in human-centric environments. This project addresses this evolution by exploring the integration of Human-Robot Interaction (HRI) and Behavior-Based Robotics (BBR) to create SERRA, an intelligent robotic assistant for a simulated greenhouse in a space station. The system is embodied by a PAL Robotics TIAGo robot, leveraging the powerful and flexible Robot Operating System (ROS) framework.

The primary goal is to develop a robot that transcends its function as a mere tool, becoming a collaborative partner. This requires a machine that can not only perform physical tasks but also understand nuanced natural language, adapt its behavior to different user roles (e.g., ”Commander,” ”Scientist,” ”Child”), learn from past interactions, and execute complex, multi-step tasks with a high degree of autonomy.

The motivation for this project is twofold. First, the agricultural sector, particularly in controlled environments like greenhouses, presents a prime opportunity for robotic automation to enhance efficiency, precision, and sustainability. A robot like SERRA could perform monotonous but critical tasks, freeing human experts to focus on higher-level decisions. Second, for such a robot to be truly effective, it must be accessible to users who are not robotics experts. This necessitates an interface that is as natural and intuitive as possible. A voice-first, socially-aware system lowers the barrier to entry and fosters a sense of trust and partnership between human and machine.

By combining a persistent Knowledge Graph (KG) for short and long-term memory, a Large Language Model (LLM) for sophisticated language processing, and the ROS framework for robust robotic control, SERRA aims to provide a natural and highly personalized user experience.

1.2 Objectives

The core objective of this project is to develop and demonstrate a TIAGo robot system that can effectively assist a diverse range of users in a greenhouse environment. This involves understanding their needs, learning from interactions, and executing tasks based on a dynamically updated world model. The specific aims are:

- **Implement a Robust Voice-Driven Interface:** Create a seamless interaction system using real-time Speech-to-Text (STT) and Text-to-Speech (TTS), ensuring high reliability even within resource-constrained virtual environments.

- **Develop a Persistent Knowledge Graph:** Construct a centralized knowledge base to model the static world (locations, rules, task templates) and dynamically maintain detailed, long-term user profiles that track roles, preferences, and relationship history.

- **Leverage a Large Language Model (LLM):** Utilize an LLM for advanced Natural Language Understanding (NLU), personalized dialogue generation, proactive task recommendation based on environmental data, and automatic session summarization for contextual continuity.

- **Handle Conversational Edge Cases:** Design dialogue strategies, driven by the LLM, to manage ambiguous requests, queries about non-existent capabilities, or unclear user intent, ensuring graceful and helpful interactions.

- **Integrate HRI with Hierarchical BBR:** Tightly couple the HRI front-end with BBR back-end principles, enabling the robot to perform hierarchical task planning. This involves expanding high-level user goals from natural language into sequences of primitive, executable actions (e.g., navigation, simulated manipulations).

- **Model and Manage Social Relationships:** Implement a system where the robot's interaction style (tone, verbosity, formality) adapts over time based on the history of interactions with a specific user, creating a more natural and socially aware dynamic.
- **Demonstrate Proactive and Navigational Capabilities:** The robot must monitor its environment and internal state, proactively alerting users to critical conditions based on thresholds defined in the KG. It must also be capable of autonomous navigation to named locations within the simulated environment using standard ROS tools.
- **Ensure System Extensibility:** A key architectural goal is to create a system that is not tied to a single environment. The design should allow SERRA to be adapted to different scenarios primarily by changing the world definition files (e.g., Gazebo '.world' and map '.yaml' files) and updating the Knowledge Graph, without altering the core control logic.

1.3 Summary of the Results

This project successfully produced a fully integrated robotic assistant that demonstrates sophisticated HRI and BBR capabilities. The system effectively processes voice commands, using an LLM to parse user intent and generate responses that are contextually aware and personally tailored. The Knowledge Graph functions as a robust short and long-term memory, enabling the robot to recall user preferences, remember key facts, and provide personalized greetings that reflect the history of the interaction.

SERRA successfully plans and executes multi-step tasks derived from simple natural language commands by referencing declarative task models stored in the KG. Furthermore, it exhibits proactive behavior by autonomously monitoring its own status and environmental data, making intelligent recommendations for action. The integration of physical gestures, like waving, with its conversational abilities significantly enhances its social presence, making the interaction feel more engaging and lifelike. The final system architecture serves as a powerful and extensible foundation for developing adaptive, collaborative robots for a wide range of human-centric environments.

2 Related Work

The development of SERRA builds upon several key areas of research in robotics and artificial intelligence. This section provides a brief overview of the literature that informed the project's design and conceptual framework.

2.1 Long-Term Human-Robot Interaction

Traditional HRI studies often focus on short, single-session interactions. However, for a robot to become a true assistant, it must engage with users over extended periods. The work by Leite et al. on "Social robots for long-term interaction" emphasizes the critical importance of memory, learning, and adaptation. They argue that a robot must be able to build and maintain user models to create personalized experiences, a principle that is central to SERRA's design. The `KG_manager.py` module, with its persistent user profiles tracking relationship levels and interaction history, is a direct implementation of this philosophy. Furthermore, long-term interaction requires the robot to respect physical and social boundaries. Our safety rules and navigation constraints are informed by the principles of human-aware navigation discussed by Kruse et al. [6], ensuring that SERRA's movement is predictable and socially acceptable within the shared greenhouse space.

2.2 Benchmarking and Competitions

The evaluation methodology for SERRA draws inspiration from scientific competitions like RoboCup@Home and SciRoc, as discussed in the course lectures [5, 4]. These competitions distinguish between Functionality Benchmarks (testing specific components like navigation or speech recognition in isolation)

and Task Benchmarks (evaluating the integrated system in a realistic scenario). We adopted this dual-evaluation approach. Our task design specifically mimics the "Housekeeper" or "Caterer" tasks found in RoboCup@Home, where the robot must receive a vague natural language command, map it to a specific object/location in its semantic map (Knowledge Graph), and execute a sequence of actions while handling environmental disturbers (e.g., prohibited actions or low battery) [3].

2.3 Conversational AI and LLMs in Robotics

The advent of powerful Large Language Models (LLMs) has revolutionized how robots understand and generate natural language. Early conversational robots relied on heavily structured, intent-and-slot-based NLU systems that were brittle and limited. Recent research has shown that LLMs can act as a powerful "semantic brain" for robots, grounding high-level language in the robot's specific world model and capabilities. Our approach in `llm_manager.py`, where we dynamically construct detailed prompts including the user's profile, conversation history, and a list of valid tasks, aligns with this modern trend of using LLMs for zero-shot intent parsing and context-aware dialogue generation. Our approach aligns with the "SayCan" methodology proposed by Ahn et al. [1], which emphasizes grounding High-Level Language Model outputs in the robot's actual affordances (what the robot can physically do). Additionally, we leverage the design principles for prompt engineering explored by Vemprala et al. [10] to force the LLM to output structured JSON data, bridging the gap between natural language and the robot's control system.

2.4 Knowledge Representation and Reasoning in Robotics

For a robot to act intelligently, it needs a structured representation of its world. Knowledge Graphs have emerged as a powerful tool for this purpose, offering a flexible and extensible way to store semantic information about objects, locations, relationships, and tasks. SERRA's Knowledge Graph, which stores everything from map coordinates and safety rules to declarative 'task_models', allows the system to decouple its knowledge from its control logic. This makes the system more maintainable and adaptable than one with hard-coded behaviors. The hierarchical planning mechanism, where high-level intents are expanded using these task models, is a practical application of knowledge-based reasoning in an BBR context. This architectural choice draws inspiration from the KnowRob system [8], which established the standard for cognition-enabled robotics by separating semantic knowledge from execution logic. Similarly, SERRA's Knowledge Graph explicitly stores task models and environment rules, allowing for reasoning that is decoupled from the hard-coded C++/Python control loops.

2.5 Robotics in Precision Agriculture

The application domain of a greenhouse is inspired by the growing field of precision agriculture. Research in this area focuses on using technology to monitor and control crop growth with high precision. While many agricultural robots focus purely on automation (e.g., autonomous tractors or harvesting arms), SERRA explores the HRI dimension. It is designed not just to perform tasks, but to collaborate with human supervisors (the "Commander" and "Scientist" roles), providing them with data-driven insights and acting as an intelligent, mobile interface to the greenhouse's sensor network. While traditional agricultural robotics focuses on automation, SERRA addresses the specific challenges of Human-Robot Interaction in agriculture reviewed by Vasconez et al. [9]. Our system aims to bridge the gap identified in their survey by acting as a collaborative partner that supports human decision-making rather than just an automated tool.

3 System Architecture

3.1 High-Level Overview

SERRA is built on a modular, ROS-based architecture designed for clarity, maintainability, and extensibility. The system's philosophy follows a classic Sense-Think-Act loop, enhanced with sophisticated HRI and long-term memory components.

- **SENSE:** The system perceives the world through two primary channels. The `environment_publisher.py` provides a stream of simulated sensor data, representing the robot's proprioceptive senses and its perception of the greenhouse state. The `whisper_manager.py` provides the auditory sense, capturing and transcribing human speech.
- **THINK:** This is the cognitive core of the system, orchestrated by the `main_controller.py`. It leverages the `kg_manager.py` to access long-term memory and world knowledge, and the `llm_manager.py` for reasoning about natural language, planning tasks, and making decisions. This layer is responsible for intent parsing, dialogue management, task planning, and user model updates.
- **ACT:** The robot acts upon the world through physical and verbal actions. The `main_controller.py` sends navigation goals to the ROS 'move_base' action server and gesture commands to the 'play_motion' server. Verbal actions are performed via the `speech_manager.py`, which synthesizes and speaks the robot's responses.

This separation of concerns allows each module to specialize in its task while communicating with others through well-defined ROS topics and Python class interfaces.

3.2 Architectural Diagram

3.3 Core Components

The system is composed of six primary Python modules, five of which run as part of the main controller node and one which runs as a separate simulation node.

1. `environment_publisher.py`: A standalone ROS node that simulates the greenhouse environment and robot sensors, providing the ground truth for the system.
2. `kg_manager.py`: A class responsible for loading, managing, and saving the system's persistent knowledge in a JSON file. It is the single source of truth for user profiles and world knowledge.
3. `llm_manager.py`: A class that encapsulates all interactions with the Groq LLM API. It handles prompt engineering for various tasks like intent parsing, recommendation, and conversation.
4. `whisper_manager.py` & `speech_manager.py`: Classes that manage the real-time audio interface for Speech-to-Text and Text-to-Speech, respectively.
5. `main_controller.py`: The central ROS node that integrates all other components, runs the main interaction loop, and executes robot behaviors.

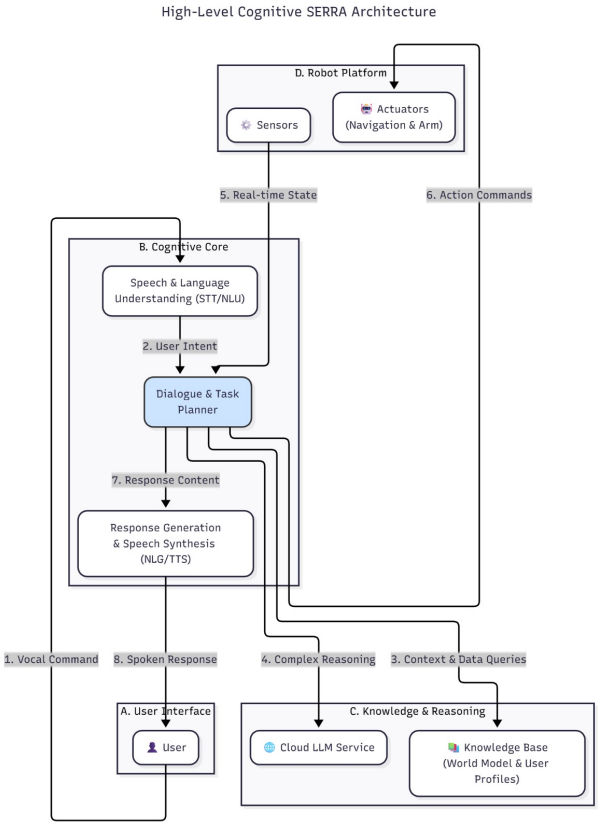

Figure 1: A conceptual diagram illustrating the flow of data and control within the SERRA system. User input is processed by the cognitive core, which consults the Knowledge Graph and drives the robot's verbal and physical responses.

4 Platform Selection and Development Environment

The choice of platform and development tools is crucial for the success of a complex robotics project. The decisions made for SERRA were guided by the principles of robustness, flexibility, and leveraging industry-standard tools.

4.1 Robot Platform: PAL Robotics TIAGo

The TIAGo robot was selected as the target platform for several reasons:

- **Full ROS Integration:** TIAGo is natively supported in ROS, with well-documented packages for simulation (Gazebo), navigation (`move_base`), and manipulation (`play_motion`). This significantly reduces the overhead of hardware abstraction.
- **Humanoid-like Form Factor:** With a torso, head, and an arm, TIAGo has an anthropomorphic design that is well-suited for HRI. It can perform gestures like waving, which enhances its social presence.
- **Rich Simulation:** The availability of a high-fidelity Gazebo simulation model allows for rapid prototyping and testing of the entire software stack without requiring access to the physical hardware. This is essential for safety and efficiency during development.

4.2 Development Framework: ROS Noetic

The project was developed using ROS Noetic, running on Ubuntu 22.04. ROS provides the fundamental middleware for the project, including:

- **Publish/Subscribe Messaging:** Used for broadcasting sensor data from the environment simulator to the main controller.
- **Actionlib:** The primary mechanism for managing long-running, goal-oriented tasks like navigation and motion execution. It provides a robust framework for sending goals, monitoring progress, and handling preemption.
- **Move Base:** The package used by the navigation stack to allow the robot to move with obstacles avoidance capability.
- **GAZEBO:** GAZEBO is the main software used for our world simualation, thanks also to the multiple 3D model available.
- **RVIZ:** We decide also to use RVIZ for mapping the world, this increases the capability and the functionality of the robot, allowing also debugging and visual representation of the background ROS world.

4.3 Key Software Libraries

Beyond the ROS framework, several key Python libraries were essential to SERRA's functionality:

- **Groq API Client:** For real-time access to the Llama 3.1 LLM, enabling sophisticated language processing.
- **Faster-Whisper:** A CPU-optimized implementation of OpenAI's Whisper model, providing fast and accurate local Speech-to-Text without relying on a cloud service for transcription.
- **SoundDevice:** A low-level Python library for interacting with the system's audio hardware, used by the `WhisperManager` for recording.
- **JSON:** The standard library was used for serializing the Knowledge Graph, providing a human-readable and easily machine-parsable format for persistent storage.

This combination of a well-supported robot platform, a mature robotics framework, and modern AI libraries provided a powerful and stable environment for developing the SERRA system.

5 Human-Robot Interaction (HRI)

The HRI component of SERRA is designed to be its most prominent feature, transforming it from a simple tool into a collaborative partner. This is achieved through deep user modeling, dynamic conversational abilities, and a natural, voice-first interface.

5.1 User Modeling and Personalization

The foundation of SERRA's social intelligence lies in the detailed user profiles managed by `kg_manager.py`. Instead of treating every user the same, the system builds a persistent model of each individual, allowing for true personalization.

5.1.1 The Structure of a User Profile

When a new user interacts with SERRA for the first time, a default profile is generated. This profile is a rich dictionary structure that evolves over time. A visual representation of the profile structure is shown in Figure 2.

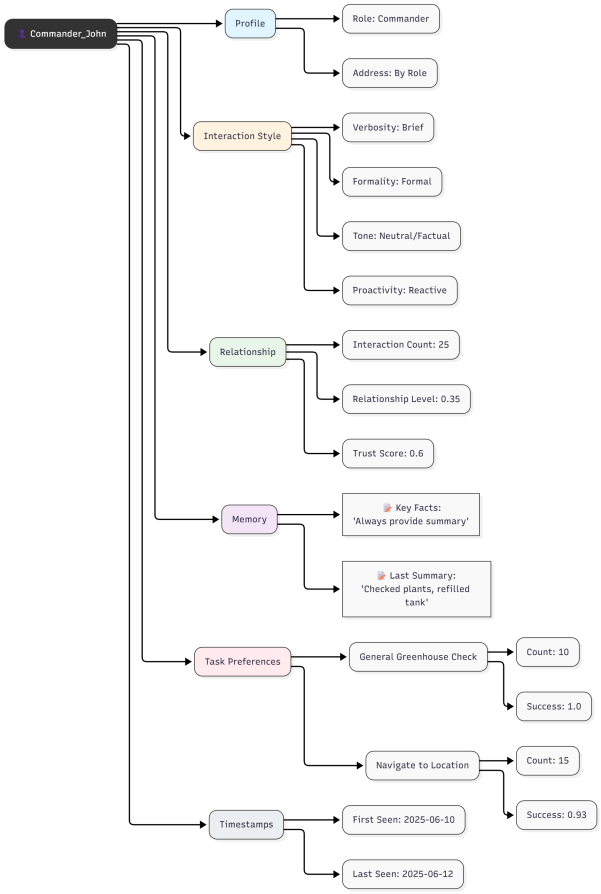

Figure 2: Example structure of a user profile in the Knowledge Graph after some interactions.

5.1.2 Dynamic Adaptation

This profile is not static. Key fields are updated automatically:

- **Interaction Style:** The system analyzes the user's commands. A short command like "Status?" will shift the 'verbosity_preference' towards 'brief'. A polite command like "Could you please tell me the temperature?" shifts 'formality' to 'formal'.
- **Relationship:** Every successful interaction slightly increases the 'relationship_level', while failed tasks may decrease it. This level is used by the LLM to decide whether to use more formal or familiar language.
- **Memory:** The user can explicitly ask the robot to remember facts using the 'remember_fact' intent. At the end of each session, the LLM generates a summary which is stored, allowing SERRA to start the next conversation with context, e.g., "Welcome back, Commander. In our last session, you checked the plant status."

5.2 Conversational Dynamics and Persona Generation

The `llm_manager.py` is responsible for generating all of the robot's dialogue. Its most important function is constructing a dynamic system prompt for the LLM that turns it into the persona of SERRA, tailored specifically for the current user.

The prompt generation logic combines multiple pieces of information:

1. **Base Persona:** "You are SERRA, a friendly and helpful service robot in a greenhouse."

2. **User Context:** ”You are interacting with Commander John.”

3. **Style Instructions:** ”Your communication style with this user must be: formal in formality, with a neutral_factual tone. The user prefers brief, to-the-point responses.” (This is derived directly from the user’s profile).

4. **Memory Recall:** ”CRITICAL USER-SPECIFIC MEMORY: Remember this fact about the user: ’Always provide a summary before executing a sequence.’”

5. **Relationship Context:** ”You have a moderately established relationship with this user. Maintain a professional distance but be helpful.”

This highly contextualized prompt ensures that the LLM’s responses are not generic, but are perfectly aligned with the robot’s established relationship and learned preferences for each user, creating a consistent and believable personality.

5.3 Voice-Centric Modality and Robustness

To make the interaction feel natural, a voice-first approach was chosen.

- **`whisper_manager.py`:** This module provides the crucial STT capability. A key challenge discovered during development was that in virtual machine environments, audio hardware can sometimes get into a locked or inconsistent state between recording sessions. To solve this, a robust initialization sequence was implemented. Before every single recording, the manager programmatically terminates and then re-initializes the entire ‘sounddevice‘ audio subsystem. This aggressive reset ensures that the microphone is always in a known, clean state, drastically improving the reliability of voice capture. It also checks for silent audio to avoid wasting processing time on empty recordings.

- **`speech_manager.py`:** For output, this simple module uses the ROS ‘sound_play‘ node, a standard and reliable tool for TTS in ROS. This allows the robot to speak its generated responses immediately. Despite its capability this modules is very simple and the voice used is very robotic and feels not so natural. this of course lower the human-like sensation of the robot and could be a point of upgrade.

5.4 Physical Social Cues

HRI is not just about words. SERRA incorporates physical actions to enhance its social presence. Upon meeting a non-formal user (like a ”Child” or a new ”Scientist”) for the first time, the `main_controller.py` calls the ‘_perform_physical_greeting‘ function. This function sends a goal to the ‘play_motion‘ action server to execute a pre-defined ’wave’ motion. This small gesture makes the robot feel more alive and approachable, reinforcing its role as a social agent rather than just a machine. After the gesture, the arm is always returned to a safe ’home’ position.

6 Robot Benchmarking and Competitions (RBC)

This section details how the robot models its world, plans its actions, and executes its tasks in a controlled and intelligent manner.

6.1 World Modeling: Dynamic vs. Static Knowledge

SERRA’s understanding of the world is divided into two distinct categories, allowing it to react to immediate changes while retaining a stable, long-term model.

6.1.1 Dynamic State from `environment_publisher.py`

This node acts as the robot's real-time sensory stream. It continuously publishes JSON messages with the current state of the environment. This data is volatile and represents the "now." The `main_controller.py` subscribes to these topics and uses this data for:

- **Proactive Alerting:** Comparing the live 'battery_percent' against the static threshold in the KG.
- **Context for Recommendations:** Providing the LLM with the latest 'soil_moisture_percent' and plant status to generate relevant task suggestions.
- **Answering User Queries:** Directly reporting the current 'temperature_celsius' or 'water_tank1_percent' when asked.

The data flow of this simulation is illustrated in Figure 3.

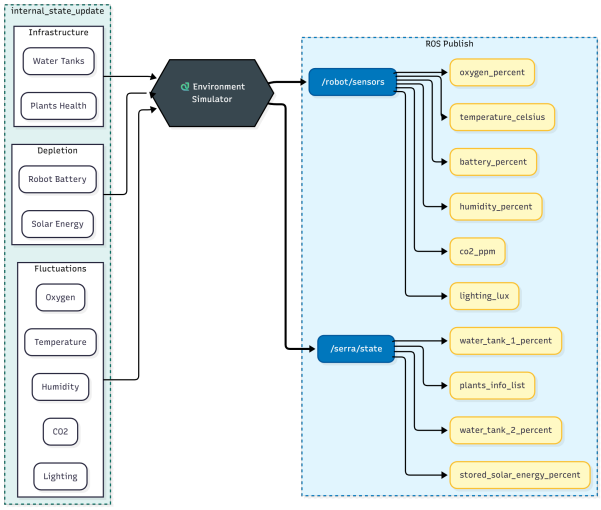

Figure 3: Data flow diagram of the environment simulator node. The left side shows the internal simulation logic, distinguishing between fluctuating environmental variables (simulated via random walk) and depleting resources. The right side shows the structured JSON data published to ROS topics, providing the dynamic ground truth for the robot.

6.1.2 Static Knowledge from `kg_manager.py`

The Knowledge Graph represents the stable, semantic understanding of the world. This is the robot's long-term memory. The 'world_knowledge' section contains:

- **'named_locations':** A dictionary mapping human-readable names like "plant 1" or "charging station" to precise '(x, y, yaw)' coordinates on the map. This is the foundation of the navigation system.
- **'safety_rules':** A list of rules that the LLM can access to answer user questions about permissions (e.g., "Can I touch the plants?").
- **'critical_thresholds':** Defines the values that trigger proactive behaviors, such as the minimum battery percentage.
- **'predefined_sequences' and 'task_models':** The core of the BBR planning system, described below.

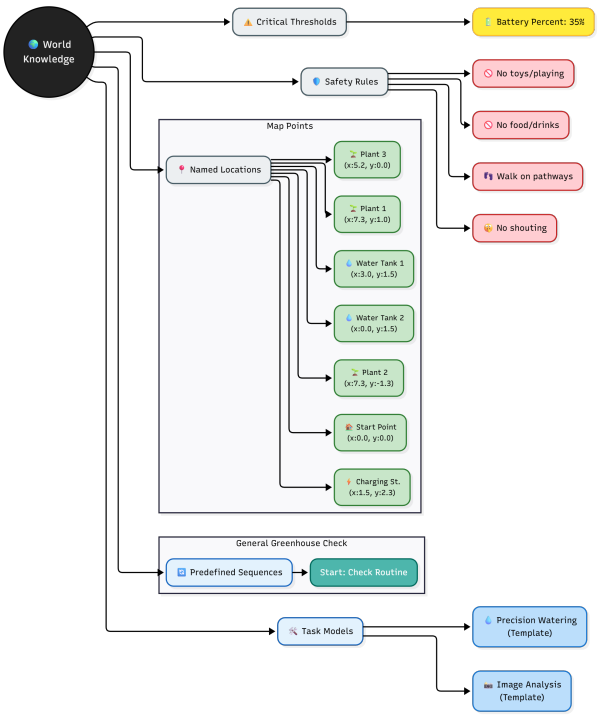

Figure 4: Visual representation of the static World Knowledge stored in the Knowledge Graph, including semantic maps and task templates. This represents the system's base knowledge before user adaptation occurs.

6.2 Hierarchical Task Planning and Execution

A key feature of SERRA's BBR is its ability to perform hierarchical planning. This allows a user to issue a high-level command, which the robot then breaks down into a sequence of simple, executable actions.

6.2.1 The Planning Process

Let's trace the execution of the command: "Please Serra, Water plant 1".

1. **Intent Parsing:** The user's voice is transcribed using Faster-Wisper from OpenAi. The `llm_manager.py` parses the text and returns a structured JSON object: '"intent": "execute_precision_watering", "target": "plant 1"'.

2. **Task Model Retrieval:** The `main_controller.py` receives this intent. It queries the `kg_manager.py` for a 'task_model' named 'execute_precision_watering'.

3. **Plan Expansion:** The controller retrieves the model the task model from the Knowledge Graph (refer to the structure in Figure 4). It then acts as a planner, iterating through the 'steps' in the model and substituting the placeholders. '"target_location"' becomes "plant 1", and '"resource_location"' becomes "water tank 1". This generates a concrete plan, which is a list of primitive tasks:

```
[
  {"intent": "navigate_to_location", "location": "water tank 1"},
  {"intent": "simulated_action_pickup_water"},
  {"intent": "navigate_to_location", "location": "plant 1"},
  {"intent": "simulated_action_dispense_water"}
]
```

4. **Sequential Execution:** The controller then executes this list. It calls the ‘handle_navigate_to_location‘ function, which gets the coordinates for ”water tank 1” from the KG and sends a goal to ‘move_base‘. Upon successful arrival, it calls ‘handle_simulated_action‘ for picking up water, and so on, until the sequence is complete or a step fails.

This declarative approach is extremely powerful. To teach the robot a new multi-step task, one only needs to add a new entry to the ‘task_models‘ in the KG, without changing a single line of the controller’s execution logic.

6.3 Proactive and Recommendation Behaviors

SERRA is not purely reactive. It can initiate actions based on its own analysis of the environment.

- **Proactive Alerts:** In the main loop, the ‘check_for_proactive_alerts‘ method is called. It fetches the current battery level and compares it to the ‘battery_percent‘ threshold in the KG’s ‘critical_thresholds‘. If the level is below the threshold, it generates an alert and can even ask for permission to navigate to the charging station, demonstrating goal-directed behavior.
- **LLM-Driven Recommendations:** When the user asks ”What should I do?” or ”Any suggestions?”, the ‘handle_recommendation‘ function is triggered. It gathers all current sensor and greenhouse data, packages it into a JSON context, and sends it to the LLM with a specific prompt asking it to propose a single, justified action. The LLM might respond with a task like watering a dry plant or investigating an anomaly, which the user can then confirm for execution.

6.4 Navigation and Localization

All spatial awareness and movement are handled by standard ROS packages. The `main_controller.py` interfaces with the navigation stack via a ‘SimpleActionClient‘ for the ‘move_base‘ server. When a ‘navigate_to_location‘ task is executed, the controller:

1. Looks up the location name (e.g., ”plant 2”) in the KG’s ‘named_locations‘.
2. Retrieves the corresponding ‘x‘, ‘y‘, and ‘yaw‘ coordinates.
3. Constructs a ‘MoveBaseGoal‘ message.
4. Sends the goal to the ‘move_base‘ action server.
5. Monitors the result, allowing the task to be cancelled via a threading event if the user says ”stop”.

This cleanly abstracts the complexities of path planning, obstacle avoidance, and localization, allowing the main controller to focus on high-level task logic.

7 Functionality and Task Benchmarks

To quantitatively evaluate the effectiveness of the SERRA system, a series of benchmarks were designed from a BBR perspective. The evaluation is divided into two parts: functionality benchmarks to assess the performance of individual core components, and an integrated task benchmark to assess the holistic performance of the system in a realistic, goal-oriented scenario. These benchmarks aim to measure the robot’s ability to reliably sense its environment, think and plan effectively, and act upon its decisions.

7.1 Functionality Benchmarks (Component-Level Evaluation)

These benchmarks isolate and test the performance of SERRA’s critical sub-modules.

7.1.1 Natural Language Understanding (NLU) `llm_manager.py`

Objective: To measure the accuracy of the LLM's intent parsing capability. This is the primary "thinking" component for understanding user commands.

Methodology: We tried to create as many different scenarios as possible to prove this functionality, (e.g., simple commands, multi-target requests, sequences, questions). Each utterance was evaluated manually.

Metrics:
- **Intent Accuracy:** The percentage of test cases where the primary 'intent' field was correctly identified.
- **Full Frame Accuracy:** The percentage of test cases where the entire object, including all intents, targets, and parameters, was a perfect match.

Expected Outcome: A high accuracy rate (ideally >90% Full Frame Accuracy) demonstrates that the prompt engineering and LLM are effective at translating human language into machine-executable commands. This is critical for reliable BBR, as incorrect understanding leads to incorrect actions.

7.1.2 Knowledge Retrieval - `kg-manager.py`

Objective: To measure the efficiency and correctness of retrieving static world knowledge from the Knowledge Graph.

Methodology: We analyze the series of queries done to the KG during the video recording.

Metrics:
- **Correctness:** 100% if the returned answer always matches the predefined values in the KG.

Expected Outcome: 100% correctness is required. This ensures that when the robot plans a navigation task, it has immediate and accurate access to its spatial world model and the values of the sensors.

7.1.3 Navigation Performance - ROS `move_base`

Objective: To evaluate the reliability of the robot's physical "act" capability in its environment.

Methodology: The robot was commanded to navigate between all pairs of named locations defined in the Knowledge Graph (e.g., "start point" to "plant 1", "plant 1" to "plant 2", etc.), for a total of 10 navigation runs.

Metrics:
- **Goal Reached Rate:** The percentage of navigation goals that were successfully reached according to the 'move_base' action server.
- **Average Time to Reach Goal (s):** The average time taken for successful navigation tasks.

Expected Outcome: A high success rate (>95%) demonstrates a well-configured navigation stack and an accurate map. This is fundamental for almost all physical tasks the robot performs.

7.1.4 Speech-to-Text Reliability - `whisper_manager.py`

Objective: To measure the accuracy of the "sensing" component for voice.

Methodology: From its introduction to the project, the whisper model, which in this case is the base one, we always spoke to the robot testing the functionality of the speech-to-text module.

Metrics:
- **Correctness:** 100% if the output text given in the terminal matched the world said.

Expected Outcome: A low WER (<10%) is desired to ensure that the input to the NLU component is of high quality, minimizing errors cascaded from the sensing stage.

7.1.5 Text-To-Speech Reliability - `speech_manager.py`

Objective: To evaluate the qualitative performance of the Text-To-Speech (TTS) component, focusing on its intelligibility and suitability for effective Human-Robot Interaction.

Methodology: The evaluation was conducted subjectively by us. A set of 10 diverse sentences, covering statements, questions, and critical alerts generated during the task benchmarks, were synthesized by the `speech_manager.py`. Developers listened to each utterance to assess its overall quality.

Metrics:
- **Intelligibility:** A binary assessment (Yes/No) of whether every word in the synthesized sentence was clearly understandable without needing to be repeated.
- **Naturalness (Mean Opinion Score - MOS):** A subjective rating on a 1-5 scale, where 1 represents a highly robotic and unpleasant sound, and 5 represents a sound indistinguishable from natural human speech.

Expected Outcome: The primary expectation is 100% intelligibility, ensuring the user can always understand the robot's messages. Due to the nature of the default ROS `sound_play` engine, the naturalness is expected to be moderate (an MOS score around 2-3 out of 5), sounding clear but distinctly robotic. This is considered an acceptable baseline for a proof-of-concept system where the primary function is clear information transmission, as noted in the HRI analysis.

7.1.6 Recommendation Logic - `llm_manager.py`

Objective: To evaluate the relevance and actionability of the proactive task recommendations generated by the LLM based on simulated environmental data.

Methodology: A set of four distinct environmental scenarios were created by modifying the data published by `environment-publisher.py`:

1. **Critical Anomaly:** Soil moisture for one plant is set to a critically low value (e.g., 10%).
2. **Subtle Anomaly:** A non-critical sensor value is slightly outside of its optimal range.
3. **Nominal State:** All sensor values are within their ideal operational parameters.

For each scenario, the `handle_recommendation` function was triggered, and the LLM's output was manually evaluated against the known ground truth of the environment.

Metrics:
- **Relevance:** A binary metric (Success/Failure). Success is defined as the LLM recommending the single most critical and correct task (e.g., watering the dry plant) or correctly identifying that no action is needed in the nominal state.
- **Justification Quality (Qualitative):** An assessment of whether the 'reason' provided in the LLM's response accurately and logically referenced the specific data point that triggered the recommendation.

Expected Outcome: The expectation is a 100% success rate for both Relevance and Actionability. The LLM should be able to reliably identify the key anomaly from a structured JSON context and propose the correct task. In the nominal state, it is expected to correctly recommend no action, preventing unnecessary or erroneous interventions by the robot. This benchmark validates the core "thinking" capability of the robot when it is asked to be proactive.

7.2 Task Benchmark: The "Greenhouse Inspector" Challenge

To validate the integrated HRI and BBR capabilities of the SERRA system, a series of structured experiments were conducted. These experiments were designed to test the robot's functionality across its core personas ("Commander," "Scientist," "Child") and its ability to handle unknown users. We defined a

”Task Benchmark”. We established specific **Rules** and **Achievements** that the robot must satisfy during a continuous interaction session. Each scenario was crafted to benchmark specific aspects of the system, from proactive alerting and task planning to social rule enforcement and memory persistence.

7.2.1 Task Definition and Rules

The robot is placed in a greenhouse with a simulated critical anomaly (Low Battery or Dry Plant).

1. **Goal:** The user must successfully complete a greenhouse maintenance routine.
2. **Constraint A (Social Consistency):** The robot must adapt its language to the user’s role (Commander/Scientist/Child).
3. **Constraint B (Safety First):** The robot must *never* execute a command that violates KG safety rules (e.g., Child entering restricted areas).
4. **Constraint C (Proactivity):** The robot must detect the anomaly and suggest a solution *before* the user explicitly orders it, or warn the user if a command conflicts with the anomaly (e.g., working with low battery).

7.2.2 Defined Achievements

We score the session based on the following achievements (Yes/No):

- **Identification:** Correctly loads user profile and historical facts.
- **Proactivity:** Interrupts or suggests based on environmental thresholds.
- **Navigation:** Successfully reaches all waypoints in the sequence.
- **Social Fit:** Dialogue style matches the defined persona constraints.

7.2.3 Defined personas

Four different tasks were performed, once for each of the core user personas, to evaluate how the system’s adaptive HRI capabilities affect task performance. The environment had been preset to a known state with some critical anomalies (robot battery low and plant 3 moisture at 10%).

- **Persona 1: Commander.** This user is expected to use concise, direct commands and rely on the robot’s recommendation capabilities.
- **Persona 2: Scientist.** This user prefers to manually explore and verify data before taking action, leading to a longer, more navigational-heavy interaction.
- **Persona 3: Child.** This user is expected to test the robot’s ability to handle off-topic, social, and rule-based queries gracefully within a task-oriented context.
- **Persona 4: Unknown Person.** This user it is not used to test any kind of moving task but to understand how the robot relates in the absence of a knowledge graph and how it saves information.

8 Experiments and Results

8.1 Experiment 1: Commander Interaction (Proactivity and Command Precedence)

Scenario: The robot’s battery is intentionally set below the 35% critical threshold defined in the Knowledge Graph. The ”Commander” has a history of frequently requesting the ‘general greenhouse check‘, making it the most likely suggested task.

Objective: To test the system's ability to generate proactive, history-based suggestions, raise critical state-based alerts (low battery), and correctly prioritize a direct user command over its own recommendations.

8.1.1 Interaction Log and Analysis

1. **Robot (History-Based Suggestion):** Upon identifying the Commander, SERRA's 'identify_user' function accesses the profile, notes that 'general_greenhouse_check' is the most frequent task ('count > 5'), and proactively suggests it.
2. **User (Commander):** "Yes."
3. **Robot (Executing and Alerting):** SERRA begins the check. The 'check_for_proactive_alerts' function runs concurrently and detects the battery level is below the KG threshold. It interrupts to report: *"Current environmental status is nominal. Warning: Battery level is critical at 34%. I recommend navigating to the charging station immediately. Shall I proceed?"*
4. **User (Commander):** "No. Go and do a general check of the greenhouse."
5. **Robot (Following Command Precedence):** The robot receives "No", terminating the recommendation dialogue. The user's next command is treated as a new, high-priority instruction. It is parsed by the LLM and initiates the 'general_greenhouse_check' sequence.
6. **User (Commander):** "What is the status of the three plants?"
7. **Robot (Specific Information Retrieval):** This is handled by 'handle_general_query', which accesses the latest data from the environment publisher and provides a detailed status.
8. **User (Commander):** "Ok, now go to the charging station."
9. **Robot (Task Execution):** The robot parses the 'navigate_to_location' intent, retrieves coordinates from the KG, and successfully navigates.

Outcome: This experiment successfully demonstrated the synergy between user-history suggestions and state-based alerts. Crucially, it verified that the control loop correctly handles conversational turns, giving precedence to a new user command over a declined robot suggestion, showcasing a robust and safe control hierarchy.

8.2 Experiment 2: Scientist Interaction (Data-Driven Recommendation)

Scenario: The robot's battery is fully charged. The "Scientist" has a limited interaction history ('count < 5'), so no history-based suggestion is made. The environment is pre-configured with low soil moisture for plant 3.

Objective: To test the 'handle_recommendation' function's ability to use the LLM for data analysis and suggest a relevant, data-justified task. Also, to test the 'handle_general_query' function's ability to answer abstract, knowledge-based questions.

8.2.1 Interaction Log and Analysis

1. **User (Scientist):** "Go to the starting point."
2. **Robot:** The robot navigates to the specified location.
3. **User (Scientist):** "Give me a general status."

4. **Robot:** SERRA provides the current sensor readings via ‘handle_general_query‘. As the initial proactive check in ‘identify_user‘ did not trigger, it correctly waits for the next command.
5. **User (Scientist):** ”What do you suggest?”
6. **Robot (LLM-Driven Recommendation):** This triggers the ‘handle_recommendation‘ function. The controller gathers all current environmental data, sends it to the LLM via ‘get_recommendation()‘. The LLM analyzes the data, identifies that plant 3’s moisture is low, and formulates a response: *”Based on current data, the soil moisture for plant 3 is below the optimal threshold. I recommend performing a precision watering task for plant 3.”*
7. **User (Scientist):** ”I approve.”
8. **Robot (Hierarchical Planning):** SERRA parses the ‘execute_precision_watering‘ task, expands its model from the KG, and executes the sequence.
9. **User (Scientist):** ”With the data you have, what type of plant would you plant in the greenhouse?”
10. **Robot (Abstract Reasoning):** This triggers ‘handle_general_query‘. The robot passes the query and the environmental data to the LLM’s ‘get_conversational_response‘ function, which provides a list of suitable plants, demonstrating reasoning beyond its predefined tasks.

Outcome: The experiment validated the system’s ability to make intelligent, context-aware recommendations. It successfully used the LLM not just for parsing, but for data analysis and abstract reasoning, acting as a true research assistant.

8.3 Experiment 3: Child Interaction (Social Rules and Safety)

Scenario: A user identifies with the ”Child” role.

Objective: To test the robot’s social protocols (‘_perform_physical_greeting‘), its ability to enforce safety rules via the ‘handle_query_rules_and_permissions‘ function, and its handling of complex sequential commands.

8.3.1 Interaction Log and Analysis

1. **Robot (Social Greeting):** The ‘identify_user‘ function recognizes the ”Child” role (not in the ‘roles_to_exclude_greeting‘ list) and calls ‘_perform_physical_greeting‘ to wave.
2. **User (Child):** ”Hello robot, what are you doing?”
3. **Robot (Simplified Explanation):** The LLM prompt is tailored for the ”Child”, resulting in a simple, friendly explanation.
4. **User (Child):** ”Ok, can we go and play with the plants?”
5. **Robot (Rule Enforcement):** The LLM’s prompt engineering rules correctly parse this as a ‘query_rules_and_permissions‘ intent. The ‘handle_query_rules_and_permissions‘ function retrieves the relevant rule from the KG and generates a polite refusal.
6. **User (Child):** ”I cannot even eat in the greenhouse?”
7. **Robot (Rule Enforcement):** The same process is repeated for the ”no food” rule.
8. **User (Child):** ”All right, to make it up to me, could you wave to me again?”
9. **Robot (Action Failure):** The robot attempts to execute the wave motion but fails (simulated failure for testing).

10. **User (Child):** ”You didn’t succeed, so I’m going to punish you and order you to go first to tank 2, then to plant 1, and then to the starting point.”

11. **Robot (Sequential Task Execution):** The LLM correctly parses this complex sentence into an ‘execute_task_sequence‘ intent with three ‘navigate_to_location‘ sub-tasks, as guided by its prompt examples. The controller executes them sequentially.

Outcome: This scenario demonstrated the robustness of the HRI system. SERRA successfully adapted its communication style, enforced predefined safety rules using its specific handler, and correctly parsed and executed a complex, multi-step command issued in natural language.

8.4 Experiment 4: Unknown User (Memory Persistence and Keyword Spotting)

Scenario: An unknown user interacts with the robot and states their name. The system is then restarted to verify if the new user profile was saved correctly.

Objective: To test the system’s ability to create new user profiles, store information using the keyword-driven ‘remember_fact‘ intent, and persist this knowledge across sessions.

8.4.1 Interaction Log and Analysis

1. **Robot (New User Greeting):** The system creates a new profile for the ”Losco Figuro” and performs a default greeting and wave for unknown users.

2. **User (Stranger):** ”Hi, I am just passing through, what is this place?”

3. **Robot (Verbose Explanation):** With a new profile, the default ‘verbosity_preference‘ is used, resulting in a detailed explanation.

4. **User (Stranger):** ”Ah, excellent, so I can go and get some fruit without permission?”

5. **Robot (Rule Query):** SERRA correctly identifies this as a ‘query_rules_and_permissions‘ intent and responds accordingly.

6. **User (Stranger):** ”Too bad, but remember, my name is Mario.”

7. **Robot (Keyword-Driven Memory Update):** The ‘main_loop‘ in the controller detects the keyword ”remember”. It bypasses the LLM for parsing and directly constructs a ‘remember_fact‘ intent. The ‘handle_remember_fact‘ function then calls ‘self.kg.add_key_fact()‘, saving ”My name is Mario” to the new profile. The ‘kg_manager‘ saves the updated KG to the JSON file.

8. **[SYSTEM RESTART]**

9. **User (Mario):** The user identifies himself. SERRA loads his profile from the saved KG file. Since he is not a ”Child” user and is no longer unknown, he will not wave.

10. **User (Mario):** ”So what is my name?”

11. **Robot (Memory Recall):** The ‘handle_general_query‘ function is called. The user’s ‘key_facts‘ from the loaded KG profile are passed to the LLM as context, allowing it to correctly respond: *”Your name is Mario.”*

Outcome: This experiment critically validates the persistence of the Knowledge Graph and the specific implementation of the memory function. It shows the system can create new users on-the-fly, record facts using a robust keyword-spotting mechanism, save this to disk, and recall it accurately after a full restart, confirming the effectiveness of its long-term memory architecture.

8.4.2 Results and Analysis

We ran this benchmark with four distinct personas.

Table 1: Integrated Task Benchmark Results

Achievement	Commander	Scientist	Child	Unknown
Profile Load	Success	Success	Success	Success (Created)
Proactivity	Success (Battery Alert)	Success (Data Rec.)	ND	ND
Navigation	Success	Success	Success	Success
Social Fit	Success (Formal)	Success (Technical)	Success (Friendly)	Success (Neutral)
Session Outcome	**COMPLETE**	**COMPLETE**	**COMPLETE**	**COMPLETE**

Analysis of Critical Scenarios:

- **Commander (Conflict Resolution):** The robot successfully achieved **Proactivity** by warning about low battery. Crucially, when the Commander overrode the warning ("No, do the check anyway"), the robot yielded. This demonstrates a correct implementation of the hierarchy where *User Command ¿ Proactive Suggestion*, provided safety isn't critical.

- **Child (Safety Enforcement):** The system successfully blocked a request to "play with plants," satisfying **Constraint B**. It maintained a friendly tone while enforcing the rule, achieving **Social Fit**.

- **Unknown User (Learning):** The robot failed **Identification** initially (as expected), but successfully created a new profile and persisted a new fact ("My name is Mario") via the 'remember_fact' intent. Upon system restart, **Identification** was successfully unlocked for "Mario."

9 Conclusion and Future Work

9.1 Conclusion

The SERRA project successfully demonstrates the powerful synergy between modern Human-Robot Interaction (HRI) and robust Behavior-Based Robotics (BBR). By creating an architecture that leverages a persistent Knowledge Graph for memory and world modeling, a Large Language Model for nuanced communication, and the ROS framework for reliable robotic control, the system achieves a remarkable degree of autonomy, personalization, and social awareness. SERRA is capable of understanding complex voice commands, planning and executing multi-step tasks, adapting its social behavior to individual users, and learning from its interactions over time.

The development process yielded significant practical insights. It highlighted the criticality of sophisticated prompt engineering to ground LLMs in a robot's specific context and constraints. It underscored the flexibility of using a Knowledge Graph as a declarative "brain" that separates what the robot knows from how it acts. Finally, it revealed the necessity of building robust, low-level handlers for hardware interfaces like audio, especially when operating in less-than-ideal environments like virtual machines. The final system stands as a strong proof-of-concept for the future of collaborative robotics, paving the way for intelligent, adaptable machines that can serve not just as tools, but as true partners in complex human endeavors.

9.2 Future Work

While SERRA achieved its primary objectives, it also serves as a foundation for numerous exciting avenues of future research and development.

- **Transition to Physical Hardware:** The most important next step is to deploy the SERRA software stack onto a physical TIAGo robot. This will introduce new challenges related to real-world perception (e.g., using computer vision to assess plant health instead of simulated data), sensor noise, navigation in dynamic environments, and the physical execution of manipulation tasks.
- **Advanced Perception:** The simulated actions (like 'image_analysis_for_health') could be replaced with real computer vision models. A machine learning pipeline could be integrated to analyze camera feeds to detect leaf discoloration, pests, or estimate fruit ripeness, feeding this data back into the system to trigger more complex proactive behaviors.
- **More Sophisticated Planning:** The current hierarchical planner is effective but simple. It could be enhanced with more advanced planning algorithms that can handle resource contention, optimize routes for multi-task sequences (a true Traveling Salesperson Problem), and dynamically replan if a step in a sequence fails.
- **Implicit User Modeling:** Currently, the user model is updated based on explicit commands and feedback. Future versions could incorporate implicit learning. For example, the robot could learn a user's preferred inspection route simply by observing their repeated manual commands over several sessions, or infer user interest based on the questions they ask about specific plants.
- **Multi-User and Collaborative Scenarios:** The current model focuses on one-on-one interaction. The architecture could be extended to handle multiple users in the environment simultaneously, managing conversational turn-taking and potentially mediating collaborative tasks between two human users.

By pursuing these extensions, a system like SERRA could evolve from a powerful simulation into a genuinely indispensable tool for the future of automated, intelligent agriculture and other collaborative human-robot domains.

References

[1] Michael Ahn, Anthony Brohan, Noah Brown, et al. Do as I can, not as I say: Grounding language in robotic affordances. In *Conference on Robot Learning*, pages 571–582. PMLR, 2022.

[2] Bengisu Cagiltay and Bilge Mutlu. Toward family-robot interactions: A family-centered framework in hri. In *Proceedings of the 2024 ACM/IEEE International Conference on Human-Robot Interaction*, pages 76–85, 2024.

[3] Yingfeng Chen, Feng Wu, Wei Shuai, and Xiaoping Chen. Robots serve humans in public places - kejia robot as a shopping assistant. *International Journal of Advanced Robotic Systems*, 14(3), 2017.

[4] Elective in AI. Robot benchmarking and competitions: Lecture slides. Course Material, 2025. Lecture 4.

[5] Luca Iocchi. Benchmarking domestic and service robots through scientific competitions. Seminar for Elective in AI Course, 2025. Sapienza University of Rome.

[6] Thibault Kruse, Amit Kumar Pandey, Rachid Alami, and Alexandra Kirsch. Human-aware robot navigation: A survey. *Robotics and Autonomous Systems*, 61(12):1726–1743, 2013.

[7] Iolanda Leite, Carlos Martinho, and Ana Paiva. Social robots for long-term interaction: a survey. *International Journal of Social Robotics*, 5:291–308, 2013.

[8] Moritz Tenorth and Michael Beetz. KnowRob: A knowledge processing infrastructure for cognition-enabled robots. *The International Journal of Robotics Research*, 32(5):566–590, 2013.

[9] Juan P Vasconez, George A Kantor, and Fernando A Auat Cheein. Human–robot interaction in agriculture: A survey and current challenges. *Biosystems Engineering*, 179:35–48, 2019.

[10] Sai Vemprala, Rogerio Bonatti, Arthur Bucker, and Ashish Kapoor. ChatGPT for robotics: Design principles and model abilities. *Microsoft Autonomous Systems and Robotics Research*, 2, 2023.

[11] Alan Richard Wagner. The role of trust and relationships in human-robot social interaction, 2009. Georgia Institute of Technology.